Text-to-SQL con QLoRA

Traducción reproducible de preguntas en español a MySQL

Objetivo

Ajustar Qwen2.5-Coder-1.5B-Instruct mediante QLoRA y medir exactitud de ejecución sobre esquemas inéditos, conservando una ruta de inferencia local, determinista y auditable.

Wilson Alejandro Calderón Argueta 22018
Pablo Daniel Barillas Moreno 22193

Deep Learning y Sistemas Inteligentes
2026

Índice

1. Resumen **2**

2. Problema y contrato **2**

3. Datos **2**
3.1. Construcción y procedencia 2

4. Metodología experimental **2**
4.1. Modelo, cuantización y prompt 2
4.2. Comparación controlada 3
4.3. Métrica y experimento de reparación 3

5. Resultados **3**

6. Reproducibilidad y riesgos **3**

7. Conclusiones **3**

1. Resumen

Se construyó una canalización completa para generar SQL de lectura a partir de un esquema y una pregunta en español. La base experimental contiene 2,592 pares aceptados sobre 12 esquemas de entrenamiento y 40 preguntas sobre dos esquemas inéditos. Todas las consultas de referencia se ejecutaron sin error en MySQL 8.4.10 con `ONLY_FULL_GROUP_BY`. El repositorio implementa dos configuraciones QLoRA que difieren únicamente en el rango, evaluación por ejecución y una reparación opcional. Al momento de congelar este borrador, las corridas en la GPU CUDA local aún no se habían realizado; por integridad académica, sus métricas se reservan explícitamente. La GPU T4 se conserva únicamente como entorno final de compatibilidad del docente.

Resultado principal

Baseline: **Pendiente de la corrida local**. QLoRA $r = 16$: **Pendiente de la corrida local**. QLoRA $r = 32$: **Pendiente de la corrida local**.

2. Problema y contrato

La función oficial `generar_sql(esquema, pregunta, ejecutar=None)` devuelve una sola sentencia `SELECT` o `WITH ... SELECT`. No emplea APIs externas, no ejecuta operaciones mutables y usa generación determinista. `config.json` permite activar o desactivar el adapter sin cambiar el prompt. El callback `ejecutar` se usa como máximo dos veces: una prueba inicial y, solo si falla, una reparación guiada por el mensaje del motor.

La aplicación React y la API FastAPI son una capa de demostración y no forman parte de la ruta evaluada. La salida pasa por una guardia basada en análisis sintáctico que rechaza sentencias múltiples, DDL y DML.

3. Datos

3.1. Construcción y procedencia

Se diseñaron 14 esquemas relacionales sintéticos y revisables, cada uno con cinco tablas, claves foráneas, índices y datos semilla. Doce dominios se asignaron a entrenamiento; *aerolínea* y *eventos* se congelaron para validación. Las plantillas cubren filtros, orden, agregaciones, agrupación, `JOIN`, subconsultas y ventanas. La semilla global es 2026 y cada fila registra fuente, licencia, dificultad y esquema.

Cuadro 1: Manifiesto validado del dataset.

Partición	Esquemas	Ejemplos	Estado
Entrenamiento	12	2,592	Aceptados en MySQL
Validación propia	2	40	Congelados y ejecutables
Descartes	–	0	Archivo auditable

La validación propia contiene 14 preguntas fáciles, 14 medias y 12 difíciles. La separación se verifica por `schema_id`; no hay intersección entre particiones. Los hashes SHA-256 y la versión exacta del motor se conservan en `resultados/dataset_manifest.json`. La revisión cruzada de 100 filas queda reservada para los integrantes en `datos/revision_muestra.csv`.

4. Metodología experimental

4.1. Modelo, cuantización y prompt

Se usa `Qwen/Qwen2.5-Coder-1.5B-Instruct` en cuantización NF4 de 4 bits, cómputo FP16 y checkpointing de gradientes. El prompt incluye el dialecto, el esquema, la pregunta y la restricción de emitir únicamente SQL. El mismo texto se usa para baseline y adapters.

4.2. Comparación controlada

Cuadro 2: Única variable experimental.

Corrida	r	α	Constantes	Hipótesis
A	16	32	dropout .05, semilla 2026, datos y épocas	Menor costo, capacidad suficiente
B	32	32	dropout .05, semilla 2026, datos y épocas	Mayor capacidad para composición compleja

El entrenamiento separa desarrollo por esquema, guarda métricas y tokenizer junto con cada adapter y exige un smoke test antes de la corrida completa. La selección final no usa loss: gana la mayor exactitud de ejecución ponderada.

4.3. Métrica y experimento de reparación

La exactitud por ejecución (EX) compara resultados normalizados: redondeo numérico a dos decimales, columnas adicionales permitidas y orden exigido solo cuando el caso lo indica. Los pesos son 1, 2 y 3 para dificultad fácil, media y difícil. Para el adapter ganador se comparará generación directa contra una única reparación tras un error de MySQL.

5. Resultados

Cuadro 3: Tabla preparada para resultados reproducibles.

Variante	EX ponderada	Tasa ejecutable	Latencia p95
Baseline	**Pendiente**	**Pendiente**	**Pendiente**
QLoRA $r = 16$	**Pendiente**	**Pendiente**	**Pendiente**
QLoRA $r = 32$	**Pendiente**	**Pendiente**	**Pendiente**
Ganador + reparación	**Pendiente**	**Pendiente**	**Pendiente**

Las curvas de pérdida, parámetros entrenables, minutos de entrenamiento, memoria pico y EX por dificultad se insertarán desde los JSON y CSV generados por los scripts. Se conservarán resultados negativos y todos los fallos se clasificarán como sintaxis, entidad inexistente, `JOIN`, agregación, filtro o resultado distinto.

6. Reproducibilidad y riesgos

El entorno SQL se fija mediante Docker en MySQL 8.4.10. Los tres notebooks recorren datos, entrenamiento y evaluación en orden. Diecisiete pruebas locales cubren configuración, seguridad SQL, normalización de resultados, contrato oficial, API y dataset. El principal riesgo restante es el cambio de dominio entre esquemas sintéticos y evaluación oculta; se mitigará reportando EX por dificultad y una predicción conservadora, nunca sustituyendo datos faltantes por estimaciones presentadas como mediciones.

7. Conclusiones

La infraestructura, los datos y la evaluación están listos y verificados. El proyecto separa con claridad lo medido de lo todavía pendiente. Las conclusiones sobre el efecto del fine-tuning, el rango ganador y la utilidad de la reparación sólo podrán escribirse después de las corridas locales. Esta decisión mantiene trazabilidad y evita sobreinterpretar resultados inexistentes. La entrega final deberá probarse además en una T4 limpia, sin reentrenar, para reproducir el entorno del evaluador.

Uso de inteligencia artificial

OpenAI Codex apoyó el análisis del enunciado, la implementación, la estructura documental y la revisión técnica; v0 se utilizó como punto de partida visual. Los integrantes deben revisar los artefactos, completar la revisión humana y ser capaces de explicar el código. Ninguna API externa participa en la generación SQL evaluada.